

数据结构讲义

qwaszx

2026 年 4 月 11 日

目录

第一章 基础数据结构	2
1.1 vector	2
1.1.1 基本操作	2
1.1.2 常见问题	2
1.1.3 实现	2
1.2 链表	3
1.2.1 基本操作	3
1.3 栈和队列	3
1.4 双端队列	3
1.4.1 基本操作	3
1.4.2 实现	3
1.4.3 维护信息	4
1.5 二叉堆	4
1.5.1 基本操作	4
1.6 扩展类问题	4
1.7 单调队列和单调栈	5
1.7.1 基本功能	5
1.7.2 例题	6
1.7.3 悬线法	6
1.7.4 困难的例子	6
1.8 总结	6
第二章 静态区间数据结构	7
2.1 信息与合并	7
2.2 静态区间数据结构	10
2.2.1 幂等律与 ST 表	10
2.2.2 分治与二区间合并	11
2.3 线段树	14
2.4 * 一般的静态区间数据结构	16

1 基础数据结构

- 加深对于 `vector`、栈、队列、链表、堆等数据结构的理解
 - 识别常见的可以使用这些数据结构解决的问题
 - 强化数据结构思维：抽象问题与数据结构的对应
- 提到复杂度时默认指最坏复杂度

1.1 `vector`

1.1.1 基本操作

`vector` 支持

- 均摊 $O(1)$ 的末尾插入删除
- $O(1)$ 的随机访问¹
- $O(\text{size} - i)$ 加均摊 $O(1)$ 在第 i 个位置插入删除
- $O(\text{max size})$ 的空间

参考：<https://cppreference.cn/w/cpp/container/vector>

1.1.2 常见问题

在使用时，需要注意一些陷阱：

- 迭代器失效：修改后，迭代器可能变得不可用
- 占用空间正比于历史最大 `size` 而不是当前 `size`
- `clear` 不会释放内存。可以用 `vector<T>().swap(a)` 来释放
- 大量 `push_back` 会带来很大的常数。可以使用 `resize` 一次性分配需要的长度，这样和数组基本一样快

1.1.3 实现

我们给出一种 `vector` 的实现：如果 `size` 超过了当前容量，那么把当前容量加倍，重新申请空间，把所有东西都搬过去。

每次重新分配后，容量约等于两倍 `size`。触发下一次重新分配需要 $\Omega(\text{size})$ 次修改，重新分配的代价是 $O(\text{size})$ 。所以，平均每次修改的代价是 $O(1)$ 。

问题 1.1. 如果额外要求空间线性于 `size` 而不是历史最大 `size` 呢？

解答. `size` 小于当前容量的 $1/4$ 时，把容量减半并重新分配空间（为什么是 $1/4$ 而不是 $1/2$ ？）

¹指任意访问某个 `a[i]`

1.2 链表

1.2.1 基本操作

链表支持

- $O(1)$ 在指定元素处插入删除单个元素/另一个链表（例如可以实现 $O(1)$ 拼接）
- $O(\text{位置})$ 随机访问
- $O(\text{size})$ 的空间

例 1.2. 洛谷 P10466 邻值查找 / 洛谷 P1168 中位数

插入往往需要平衡树之类的数据结构,但是如果倒着来就只需要先排序然后一直删除。如果排序可以线性,那么就能做到线性。

1.3 栈和队列

基本操作是广为人知的,不再赘述。显然,栈在功能上弱于 `vector`,因此可以用 `vector` 实现栈。而队列则不能直接用 `vector` 实现。如果队列大小已知不超过某个上界,那么可以用数组模拟循环队列。而如果想做到 $O(\text{size})$ 空间,就需要用链表或者双端队列实现

1.4 双端队列

双端队列在功能上强于 `vector` 和队列,但是实现更复杂,常数也更大。

1.4.1 基本操作

- 均摊 $O(1)$ 头尾插入删除
- $O(1)$ 随机访问

1.4.2 实现

STL 的 `deque` 即是标准的双端队列。然而,需要开很多双端队列时,要谨慎使用 STL 的 `deque`,因为一个空 `deque` 可能占用几百到几千字节的空间。

更好的实现是对顶栈。设想有两个栈,其底部粘在一起。于是其整体就成为一个两端开口的双端队列。唯一的问题出在某个栈空了还要继续删的情况。此时,我们暴力把整个序列倒出来重构即可。换句话说,我们重新把序列均分到两个栈里。

使用 `vector` 来实现栈就能支持随机访问了。如果需要很紧的空间,可以用之前提到的 $O(\text{size})$ 空间的 `vector`。

每次重构后,两个 `vector` 都有大概 $\text{size}/2$ 个元素。到下次重构之前,至少有 $\text{size}/2 + |\text{size}' - \text{size}/2| = \Omega(\text{size}')$ 次操作。重构花费 $O(\text{size}')$ 的代价,所以插入删除是均摊 $O(1)$ 的。

NOI 2022 时有许多人因为 MLE 失去了一整道题的分数。

1.4.3 维护信息

在两个 `vector` 中各自维护自己的前缀信息，这样可以 $O(1)$ 查询目前队列中跨过分断点的任意子区间的信息和。

可以以 $\sum_i |l_i - l_{i-1}| + |r_i - r_{i-1}|$ 的代价回答多个区间询问。在很多问题中，这个总代价往往可以是线性的。

例 1.3. 支持双端队列操作、每次询问做背包后体积 V_i 的结果。需要 $O(nV)$ 。

例 1.4. “不删除双指针”。询问最短的 $\gcd > 1$ 的区间。

1.5 二叉堆

1.5.1 基本操作

- $O(1)$ 查询最小值
- $O(\log n)$ 插入
- $O(\log n)$ 删除
- $O(\log n)$ Decrease-Key
- $O(n)$ 合并

评注. 理论上最好的堆——*Brodal Queue* 支持

- $O(1)$ 查询最小值
- $O(1)$ 插入
- $O(\log n)$ 删除
- $O(1)$ Decrease-Key
- $O(1)$ 合并

但是实现复杂、常数很大。

算法/数据结构总是在复杂度、实际表现、实现难度、常数、功能之间做取舍。

还有其他一些常见的堆：左偏树、斜堆、随机堆、配对堆、二项堆、斐波那契堆。

1.6 扩展类问题

从若干个初始状态开始，每个状态可以扩展出若干个后继状态，保证扩展出来的状态的权值小于原状态的权值。每次取权值最小的状态扩展，重复若干次。

通过一定的设计，很多求前 k 大/小的问题可以变成这类问题。使用堆来维护所有目前的状态，每次取出最小的一个来扩展，注意去重。扩展形成一张图，复杂度为 $O(M \log M)$ 或 $O(k \log k + M)$ （取决于堆），其中 M 为前 k 小的点邻接的边数。

如果每个点的扩展方式都是相同的，并且只有常数种扩展方式，并且扩展出来的状态的权值关于原状态的权值是单调的，那么可以使用队列替换堆来做到 $O(M)$ 。

例 1.5. 序列合并。

解答. 方法一: (i, j) 扩展出 $(i, j + 1)$, 初始状态为所有 $(i, 1)$

方法二: (i, j) 扩展出 $(i + 1, j)$ 和 $(i, j + 1)$, 但是要去重

方法三: <https://www.cnblogs.com/zkyJuruo/p/18428124>

例 1.6. 蚯蚓.

解答. 首先处理掉“除了新的以外都加 q ”, 这可以变成全局加 q 然后新的减掉 q .

这样就是 x 扩展出 $\lfloor p(x + tq) \rfloor - (t + 1)q$ 和 $(x + tq) - \lfloor p(x + tq) \rfloor - (t + 1)q$

符合我们用队列替换堆时的要求

例 1.7. 常数种非负边权的最短路. BFS 扩展时, 从 x 扩展出 $O(1)$ 种固定的新状态可以使用队列替换堆。

例 1.8. 前 k 小子集和.

解答. 考虑一个 DFS 过程, 枚举下一个选择的数在哪

然而这样会扩展出 $\Theta(n)$ 个状态

多叉树转二叉树: 左儿子右兄弟 (注意兄弟的顺序)

例 1.9. 限制大小的前 k 小子集和. 要求集合大小在 $[L, R]$ 内。

解答. 考虑怎么枚举 $\text{size} = m$ 的集合. 假设选了 $a_{i_1}, \dots, a_{i_m}$, 转为枚举 $d_j = i_j - i_{j-1}$, 只需要 $\sum_j d_j \leq n - 1$.

初始所有 $d_j = 1$, 每次枚举下一个需要增加的 d_j 即可。

同样用儿子兄弟表示法转为二叉树

评注. 这类问题有可能与 k 短路有关联, 但我不会。有兴趣的同学可以自行研究或者一起讨论一下

例 1.10. 合并果子. 由于合并的结果总是越来越大的, 可以使用队列替换堆。

评注. 这就是 *Huffman* 编码。我们有结论

- *Huffman* 编码可以最小化 $\sum_i \text{dep}_i \cdot w_i$
- w_i 的深度大约为 $\log \frac{W}{w_i}$, 其中 $W = \sum w_i$

在不均匀分治 (如树剖、点分治、分治 FFT 等) 中, 可以使用 *Huffman* 树来降低复杂度或常数, 以后会提到。

1.7 单调队列和单调栈

1.7.1 基本功能

令 $b_i = \max\{a_i, a_{i+1}, \dots, a_r\}$ 。在 r 从 1 扫到 n 的过程中, 均摊 $O(1)$ 地维护所有 b_i 。(栈中每个元素表示一段区间的 b_i)

1.7.2 例题

例 1.11. 询问端点单调的区间最值. 多次查询 $\max\{a_{l_i}, \dots, a_{r_i}\}$, 其中 l_i, r_i 都单调不降。可以强制在线（指 a 可能依赖先前的询问结果）
虽然双端队列也可以做，但是这种问题单调队列更方便。

练习 1.12. 如果 l_i 不单调呢？（允许复杂度变大一些）

解答. 单调栈上二分

例 1.13. 求前驱后继. 对每个元素，查询它左/右边第一个比它大/小的数的位置

例 1.14. 稍复杂一些的信息维护. 对每个 r ，求 $\sum_{l=1}^r (r-l+1) \cdot \max\{a_l, \dots, a_r\}$ 。强制在线。（提示：要记得单调栈中每个元素都代表了一段区间的 $\max\{a_l, \dots, a_r\}$ 。）

离线的另解. 如果允许离线，那么可以考虑换种方式计算答案：对每个 a_i ，看看它是哪些区间 $[l, r]$ 的 $\max$ 。这可以由先前提到的前驱后继来做。

例 1.15. 最值版本. 求 $\max_{l,r} (r-l+1) \cdot \max\{a_l, \dots, a_r\}$ 。

解答. 使用离线算贡献的方法是最容易的

在线对每个 r 算的话需要维护凸包。

1.7.3 悬线法

有些题给了个二维黑白网格，要求全黑的矩形/正方形的最大面积/面积和之类的东西
对每个格子计算每个方向全黑能延伸到哪，然后就变成类似例三的问题

1.7.4 困难的例子

给定 X ，求有多少个区间 $[l, r]$ 满足 $\max + \min = X$

令 $b_i = X - a_i$ ，泛化成有多少个区间 $[l, r]$ 满足 $\max\{a_i\} = \max\{b_i\}$

a, b 交替形成一个新序列，在这个新序列上扫的时候维护一个单调栈。单调栈中允许权值相同。

单调栈中形成若干个等值段，每个等值段都是若干个 a 和若干个 b 交替出现。一个等值段的贡献为开头到最后一次 ab 切换的位置。

1.8 总结

本节课的核心内容如下：

- 熟知讲过的几种线性数据结构的功能，能够把问题对应到数据结构上
- 扩展类问题的思路
- 加深对单调栈和单调队列的认识

2 静态区间数据结构

2.1 信息与合并

在讨论具体的数据结构之前，我们需要首先理解什么是「信息」以及信息的「合并」。我们以区间信息为例。给定序列 $a = (a_1, \dots, a_n)$ 以及待查询的函数 $f_{[l,r]}(a)$ ，我们首先需要确定结果的**可能集合** S ，也就是 f 的陪域。如果 S 本身就是一个巨大的集合，那么我们自然无法期望得到一个快速的算法。

例 2.1. 假设 a 的所有元素都在集合 A 中。

- $f_{[l,r]}(a) = (a_l, \dots, a_r)$ 是一个平凡的无法快速查询的函数，因为它本身就需要写出所有的值。此时， $S = \{\epsilon\} + A + A \times A + A \times A \times A + \dots$ 是一个巨大的集合。
- $f_{[l,r]}(a) = x_0 \odot a_l \odot a_{l+1} \odot \dots \odot a_r$ 是一个常见的情况，其中 $\odot$ 是一个定义在 A 上的满足结合律的二元运算。此时， $S = A$ 。
 - 尝试代入 $\odot = +, \max, \times, \gcd$ 。
- $f_{[l,r]}(a) = \bigoplus_{[i,j] \subseteq [l,r]} (a_i \odot \dots \odot a_j)$ 是另一个常见的情况，其中 $\odot$ 同上一条， $\oplus$ 是另一个定义在 A 上的满足交换律和结合律的二元运算（请注意 $\odot$ 和 $\oplus$ 在这里并不暗示分配律）。也就是说，枚举所有的子区间，子区间内做运算 $\odot$ ，然后再把所有子区间的结果用 $\oplus$ 合并。
- $f_{[l,r]}(a) =$ 区间众数的出现次数 是一个虽然 S 很小但也没有 polylog 做法的问题。

这是半群

但是在下面暗示了。

评注. 从信息论的角度看，至少需要 $\log |S|/w$ 个变量才能表示一个 S 中的元素。从 OI 的角度看， S 中每个元素都是表示成一个结构体的，那么存储 S 中的元素就需要花费结构体中的变量个数那么多的代价。

一般的 f 并不容易快速回答。但是， f 经常有一些好的性质。如果学过一些基本的线段树知识，那么可以发现我们在对询问区间进行切分，使得每个切出来的区间都被预处理过了。我们将此抽象为**可分解性**，这是最重要和常见的性质：

这是 GPT 起的名字

定义 2.2. 可分解性. 如果存在 S 上的二元运算 $\odot$ ，使得对任何 $l \leq m < r$ 都有 $f_{[l,r]}(a) = f_{[l,m]}(a) \odot f_{[m+1,r]}(a)$ ，则称 f 可分解。

练习 2.3.

1. 假设 f 的值域为 $T \subseteq S$ ，证明 $\odot$ 在 T 上满足结合律。为什么 T 以外的部分无关紧要？
2. 证明 $f_{[l,r]}(a) = f_l(a) \odot f_{l+1}(a) \odot \dots \odot f_r(a)$ 。
3. 说明为什么我们可以不失一般性地只考虑形如 $g_{[l,r]}(b) = b_l \odot b_{l+1} \odot \dots \odot b_r$ 的问题。

练习 2.4. 假设存在一个幺元 $0 \in S$ （注意这里的 0 是抽象的 0 ）使得 $x \odot 0 = 0 \odot x = x$ 。此时，我们可以避免一些边界上的麻烦。例如，我们可以设 $f_{\varnothing}(a) = 0$ ，从而在 $m = r$ 时

也有 $f_{[l,r]}(a) = f_{[l,m]}(a) \odot f_{[m+1,r]}(a)$ 。

如果不存在么元, 说明我们总是可以人为补上一个。

例 2.5.

- $f_{[l,r]}(a) = \{a_l\} \times \{a_{l+1}\} \times \cdots \times \{a_r\}$ 也是满足结合律的 (在同构意义下)! 然而, 我们显然不想用它来计算。
- $f_{[l,r]}(a) = a_l \odot \cdots \odot a_r$ 显然是可分解的。
- $f_{[l,r]}(a) = \bigoplus_{[i,j] \subseteq [l,r]} (a_i \odot \cdots \odot a_j)$ 并不直接可分解。但是, 如果 $\odot$ 和 $\oplus$ 满足分配律 $x \odot (y \oplus z) = (x \odot y) \oplus (x \odot z)$, 那么我们可以构造出一个包含 f 的可分解的函数。令

$$g_{[l,r]}(a) = \begin{pmatrix} f_{[l,r]}(a) \\ fl_{[l,r]}(a) \\ fr_{[l,r]}(a) \\ flr_{[l,r]}(a) \end{pmatrix} = \begin{pmatrix} \bigoplus_{[i,j] \subseteq [l,r]} (a_i \odot \cdots \odot a_j) \\ \bigoplus_{[l,j] \subseteq [l,r]} (a_l \odot \cdots \odot a_j) \\ \bigoplus_{[i,r] \subseteq [l,r]} (a_i \odot \cdots \odot a_r) \\ a_l \odot \cdots \odot a_r \end{pmatrix},$$

找出使得 $g_{[l,r]}(a) = g_{[l,m]} \otimes g_{[m+1,r]}$ 的运算 $\otimes$ 。(提示: $[l,r]$ 的子区间由三部分组成: $[l,m]$ 的子区间、 $[m+1,r]$ 的子区间、跨越 $[m,m+1]$ 的子区间, 最后一部分相当于由一个 $[l,m]$ 的后缀拼上一个 $[m+1,r]$ 的前缀。可以将 $(\oplus, \odot)$ 想象成 $(+, \times)$ 。)

这是环

评注. 有许多运算 $(\oplus, \odot)$ 都构成环, 也就是 $\oplus$ 交换结合、 $\odot$ 结合、 $\odot$ 对 $\oplus$ 有分配律。常见的例子有 $(+, \times)$, $(\max, +)$, $(\max, \min)$ 。这里 $\max$ 和 $\min$ 是等效的。还有很多运算相当于多维的 $\max$ 或 $\min$, 如位运算 $\&$ 和 $|$, lcm 和 gcd , 集合运算 $\cup$ 和 $\cap$ 。

另一方面, 环导出的矩阵乘法也是一个环。所以, 我们已经可以处理相当多的运算。

评注. 我们从这个例子中可以看到, 虽然原本的 f 无法直接处理, 但放到更大的结构 g 中反而就能够处理了。往小了说, 这是设计信息时的常见技巧: 补充缺少的元素, 直到结构变得封闭; 往大了说, 这是一种深刻的哲学: 发展新的概念和理论, 让困难的问题变得平凡。

「比起用锤头敲开核桃, 不如用海水把它泡软。」

现在, 所谓的「信息合并」无非就是找到相对简单的可分解的函数 f 以及对应的运算 $\odot$ 。让我们来做一些具体的练习。

练习 2.6. 对以下询问, 找到合适的能够回答询问的可分解的 f 以及对应的运算 $\odot$:

- 区间和 $a_l + \cdots + a_r$
- 区间乘积 $a_l \cdots a_r$, 在 $\mathbb{F}_p$ 下做运算 (也就是有一个全局固定的模数 p)
- 区间最值 $\max(a_l, \dots, a_r)$
- 区间最大子段和 $\max_{[i,j] \subseteq [l,r]} (a_i + \cdots + a_j)$
- 区间异或和 $a_l \oplus \cdots \oplus a_r$
- 区间方差 $\frac{1}{r-l+1} \sum_{i=l}^r (a_i - \bar{a}_{[l,r]})^2$, 其中 $\bar{a}_{[l,r]} = \frac{1}{r-l+1} \sum_{i=l}^r a_i$ 。
- 区间 k 次方和 $a_l^k + \cdots + a_r^k$
- 区间位置乘值的和 $\sum_{i=l}^r i a_i$
- 区间两两乘积的和 $\sum_{l \leq i < j \leq r} a_i a_j$
- 区间相邻乘积的和 $\sum_{l \leq i < r} a_i a_{i+1}$

这里的 $\oplus$ 是异或而不是抽象运算。

- 区间字符串哈希 $\sum_{l \leq i \leq r} a_i B^{r-i}$
- 区间 gcd $\gcd(a_l, \dots, a_r)$
- 区间最长连续 0 长度 $\max_{[i,j] \subseteq [l,r]} [\forall k \in [i,j], a_k = 0]$
- 区间做括号匹配剩下的不匹配括号
- 区间内有多少个子区间同时包含 $1, 2, \dots, k$ (可以设计 $O(k)$ 大小的信息和 $O(k)$ 完成的运算)
- 区间内只保留最值位置上的值, 然后考虑以上信息

在实现代码时, 一种方便的写法是将 S 的表示封装进结构体, 然后重载 S 上的 $+$ 运算符。然后, 在写数据结构时, 我们用 $+$ 即可完成「信息合并」。通过这种方式, 我们解耦了数据结构与其维护的信息。

代码 2.1: 信息的封装

```

1 struct Info {
2     // 表示 S 中的一个元素需要的若干变量。我们以最大子段和为例。
3     long long s, sl, sr, slr;
4     // 默认构造函数: 对应幺元 0。如果没有直接的表示, 那么可以再加一个
        bool 变量表示。
5     Info() : s(-INF), sl(-INF), sr(-INF), slr(0) {}
6     // 构造函数, 将序列的元素 x 转成 S 中的元素
7     Info(int x) : s(x), sl(x), sr(x), slr(x) {}
8
9     // 重载 + 运算符: 实现运算 \odot
10    friend Info operator + (const Info &a, const Info &b)
11    {
12        Info res;
13        res.s = max(max(a.s, b.s), a.sl + b.sr);
14        res.sl = max(a.sl, a.slr + b.sl);
15        res.sr = max(b.sr, b.slr + a.sr);
16        res.slr = a.slr + b.slr;
17        return res;
18    }
19 };
20
21 // 在数据结构中使用示例:
22 // a[u] = Info(x);
23 // a[u] = a[u << 1] + a[u << 1 | 1];

```

有时, $\odot$ 是可逆的, 也就是已知 $a \odot b$ 和 a 时可以解出 b 。此时, 我们可以通过求出 $f_{[1,r]}$ 和 $f_{[1,l-1]}$ 来解出 $f_{[l,r]}$ 。这样的问题可以通过前缀和来做到线性。

评注. 这个概念对应消去律。这和逆元、交换律都是不同的概念。

练习 2.7. 练习 2.6 中, 哪些是可逆的? 据此得到它们的线性做法。

最后一个有用的性质是**幂等律**，它是说 $a \odot a = a$ 。典型的满足幂等律的运算是最值 $\max, \min$ 以及它们的多维版本。幂等律没有前面两个性质那么有用。在同时具备结合律的情况下，幂等律允许我们把待查询的区间拆成可重叠的区间，据此减少需要预处理的区间数量。具体来说，我们有

$$\begin{aligned} \bigodot_{k=l}^r a_k &= \left(\bigodot_{k=l}^{i-1} a_k \right) \odot \left(\bigodot_{k=i}^j a_k \right) \odot \left(\bigodot_{k=j+1}^r a_k \right) \\ &= \left(\bigodot_{k=l}^{i-1} a_k \right) \odot \left(\bigodot_{k=i}^j a_k \right) \odot \left(\bigodot_{k=i}^j a_k \right) \odot \left(\bigodot_{k=j+1}^r a_k \right) \\ &= \left(\left(\bigodot_{k=l}^{i-1} a_k \right) \odot \left(\bigodot_{k=i}^j a_k \right) \right) \odot \left(\left(\bigodot_{k=i}^j a_k \right) \odot \left(\bigodot_{k=j+1}^r a_k \right) \right) \\ &= \left(\bigodot_{k=l}^j a_k \right) \odot \left(\bigodot_{k=i}^r a_k \right) \end{aligned}$$

也就是说，我们不光可以用 $f_{[l,i]}(a)$ 和 $f_{[i+1,r]}(a)$ 拼出 $f_{[l,r]}(a)$ ，还可以用 $f_{[l,j]}(a)$ 和 $f_{[i,r]}(a)$ ($j \geq i$) 拼出 $f_{[l,r]}(a)$ 。

2.2 静态区间数据结构

在本节中，我们假设询问都是 $f[l,r] = a_l \odot \cdots \odot a_r$ 的形式，其中 $\odot$ 是有结合律的可以 $O(1)$ 计算的二元运算。我们在[练习 2.3](#)中解释了为什么可以这么做。

2.2.1 幂等律与 ST 表

我们先来展示如何对于额外满足幂等律的 $\odot$ 设计简单的数据结构来回答区间询问，也就是 ST 表 (Sparse Table)

数据结构 2.8. ST 表.

预处理. 对于所有满足 $1 \leq i + 2^k - 1 \leq n$ 的 i, k ，预处理 $st_k[i] = f[i, i + 2^k]$ 。这个递推是容易的： $st_0[i] = f[i] = a_i$ ， $st_k[i] = st_{k-1}[i] \odot st_{k-1}[i + 2^{k-1}]$ 。

询问. 询问 $f[l,r]$ 时，找到最大的 k 使得 $2^k \leq r - l + 1$ 。此时， $[l, l + 2^k]$ 和 $[r - 2^k + 1, r]$ 可以覆盖整个区间 $[l,r]$ 。所以 $f[l,r] = st_k[l] \odot st_k[r - 2^k + 1]$ 。

现在来看一下复杂度。总共有 $O(n \log n)$ 个预处理的项，所以预处理时间为 $O(n \log n)$ 。每次查询只需要合并两个预处理的项，故查询复杂度 $O(1)$ 。对于寻找最大的 $k = \lfloor \log_2(r - l + 1) \rfloor$ ，可以预处理或者使用内置的 `__builtin_ctz`。

代码 2.2: ST 表

```
1 Info st[LOGN][N];
2 // O(n log n) 预处理
```

我们没有解释为什么可以假设能 $O(1)$ 计算。实际上，如果需要 $O(T)$ 计算，那么最后复杂度往往要再乘一个 T 。

翻译之后多出了一个表 (Table)。

这个函数是 GCC 私货。以前是 CCF 系列赛事原则上禁用的。

```

3 void build() {
4     lg[1] = 0;
5     for (int i = 2; i <= n; i++) lg[i] = lg[i >> 1] + 1;
6
7     for (int i = 1; i <= n; i++) st[0][i] = Info(a[i]);
8     for (int k = 1; (1 << k) <= n; k++)
9         for (int i = 1; i + (1 << k) - 1 <= n; i++)
10             st[k][i] = st[k - 1][i] + st[k - 1][i + (1 << (k - 1))];
11 }
12 // O(1) 询问
13 Info query(int l, int r) {
14     int k = lg[r - l + 1]; // 也可以用 k=__builtin_ctz(r - l + 1);
15     return st[k][l] + st[k][r - (1 << k) + 1];
16 }

```

评注. 幂等律对于更低复杂度的数据结构设计并无太多帮助。然而，通过幂等律放松限制是一个重要的思想，这在最优化类型的动态规划中有时可以降低复杂度。

练习 2.9. 通过将 $[l, r]$ 拆成 $O(\log n)$ 个 $[p_i, p_i + 2^i]$ 的不交并，说明 ST 表也可以在 $O(\log n)$ 的时间内查询不满足幂等律的信息。

练习 2.10. 说明 ST 表支持 $O(\log n)$ 的末尾插入。再拼合双端队列，我们就能支持 $O(\log n)$ 的头尾插入。

2.2.2 分治与二区间合并

现在，我们来考虑只有结合律的情况。一种想法是**分治**：将目前待处理的区间 $[l, r]$ 分为两半 $[l, m]$ 和 $[m + 1, r]$ ，那么询问有三种情况：要么完全在左边，要么完全在右边，要么跨过中点。完全在某一边的情况可以变成子问题递归处理，而对于跨过中点的询问，我们可以将其变成一个 $[l, m]$ 上的后缀询问加上一个 $[m + 1, r]$ 上的前缀询问，再用一次 $\odot$ 拼合。于是我们可以考虑预处理每个分治区间的这样的前后缀询问的结果，这样就得到了以下的数据结构：

数据结构 2.11. 二区间合并.

预处理. 从 $[1, n]$ 开始分治。假设当前的分治区间为 $[l, r]$ ，设其中点为 $m = \lfloor (l + r)/2 \rfloor$ 。如果 $l = r$ ，那么结束；否则，预处理所有 $[l, m]$ 的后缀询问的结果以及所有 $[m + 1, r]$ 的前缀询问的结果，然后递归处理 $[l, m]$ 和 $[m + 1, r]$ 。

查询. 对于查询 $[lq, rq]$ ，从 $[1, n]$ 开始分治。假设当前的分治区间是 $[l, r]$ ，中点为 $m = \lfloor (l + r)/2 \rfloor$ 。如果 $l = r$ ，那么直接返回 a_l ；如果 $[lq, rq]$ 完全在 $[l, m]$ 或者 $[m + 1, r]$ ，则递归下去处理；否则，用预处理的前后缀计算 $f[lq, m] \odot f[m + 1, rq]$ 并返回。

在 OI 圈中，这个结构经常被叫做“猫树”。这是一个乱起名字的例子，参见[此博客](#)。

现在来看一下复杂度。设长为 n 的分治区间需要花费 $T(n)$ 的时空预处理。那么左右两个子问题各自花费 $T(n/2)$ 的时空处理, 预处理前后缀询问花费 $O(n)$ 时空, 所以总的时空复杂度是 $T(n) = 2T(n/2) + O(n) = O(n \log n)$ 。对于询问, 代价是 $Q(n) = Q(n/2) + O(1) = O(\log n)$ 。

评注. 这个数据结构在询问时只使用了一次 $\odot$ 。在这个限制下, $\Omega(n \log n)$ 的预处理复杂度是必要的。

另外, 我们注意到询问的瓶颈事实上在于找到那个可以用前后缀回答的分治区间。我们将在练习中看到这个过程可以优化到 $O(1)$ 。

代码 2.3: 二区间合并

```

1  Info a[N];
2  // 每一层分治对应一层数组
3  Info pre[LOGN][N], suf[LOGN][N];
4  void build(int l, int r, int dep) {
5      if (l == r) return;
6      int m = (l + r) >> 1;
7      suf[dep][m] = a[m];
8      for (int i = m - 1; i >= l; i--)
9          suf[dep][i] = a[i] + suf[dep][i + 1];    //注意: 加号 (实际代表 \
              odot) 未必有交换律, 所以两边不能反过来
10     pre[dep][m + 1] = a[m + 1];
11     for (int i = m + 2; i <= r; i++)
12         pre[dep][i] = pre[dep][i - 1] + a[i];
13     build(l, m, dep + 1);
14     build(m + 1, r, dep + 1);
15 }
16 // 查询 [lq, rq]
17 Info query(int l, int r, int lq, int rq, int dep) {
18     if (lq <= l && r <= rq) {
19         if (l == r) return a[l];
20     }
21     int m = (l + r) >> 1;
22     if (rq <= m) return query(l, m, lq, rq, dep + 1);
23     else if (lq > m) return query(m + 1, r, lq, rq, dep + 1);
24     else return suf[dep][lq] + pre[dep][rq];
25 }
26 Info query(int lq, int rq) {
27     return query(1, n, lq, rq, 0);
28 }

```

树与区间划分

上面这个代码并没有那么显然, 它隐式地使用了一个事实: “一层” 内的区间是不交的。为了更直观的理解这个事实, 我们来考察区间划分与树的联系。对每个分治区间, 我们建立

一个结点在分治中，一个分治区间会被划分为两个子区间，我们就把这两个子区间对应的结点作为它的孩子。这样，我们会得到一个树结构。图 2.1 和 图 2.2 展示了这两个结构，它们都可以帮助我们直观地考察数据结构。

一些资料显示结点是比节点更规范的表述，虽然实际上并没有人在意。在英文中，它们都是 node。



图 2.1: 区间划分

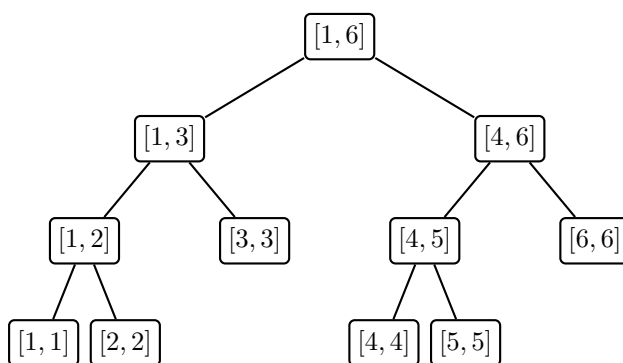


图 2.2: 分治树

我们立刻可以从图中得到以下的结论：

性质 2.12. 分治树的性质.

- 一层中的区间是不交的。
- 叶子未必具有相同的高度。
- 所有结点的区间长度之和不超过 $n \times \text{高度}$ 。
- 两个叶子的 LCA 对应了它们被划分到两侧的那个分治区间。

练习 2.13. 组合数学复习. 如果允许任意选择分治点（而不是固定为中点），那么有多少种可能的分治树？

下面的练习将询问的复杂度优化到 $O(1)$ 。

练习 2.14. 优化二区间合并的询问复杂度.

1. 假设你有一个 $O(n)$ 预处理、 $O(1)$ 查询 LCA 的算法。在这种情况下，说明询问复杂度可以达到 $O(1)$ 。
2. 即使没有这样的算法，我们也可以实现 $O(1)$ 查询。首先，在序列末尾填充一些 0 来将序列长度补齐为 2^k 。这样我们总是可以假设 n 是 2 的幂，并且不改变预处理复杂度。说明对于这样的 n ，两个叶子的 LCA 可以 $O(1)$ 计算，从而可以做到 $O(1)$ 询问。尝试用代码实现这个算法。

这种算法真的存在，不过也许有点麻烦。

- 提示一
- 提示二
- 提示三

2.3 线段树

在2.11中，我们通过预处理的方式回答了前后缀询问。那么，如果我们直接通过递归来回答前后缀询问呢？不妨考虑 $[l, r]$ 的前缀询问 $[l, i]$ 。如果 $i \leq m$ ，那么直接递归 $[l, m]$ ；否则，可以用 $f[l, m] \odot f[m+1, i]$ ，而后者是 $[m+1, r]$ 的一个前缀。我们预处理所有完整区间的 $f[l, r]$ ，询问时通过递归回答。这就得到了线段树。

数据结构 2.15. 线段树.

预处理. 从 $[1, n]$ 开始分治。假设当前的分治区间为 $[l, r]$ ，设其中点为 $m = \lfloor (l+r)/2 \rfloor$ 。如果 $l = r$ ，那么结束；否则，递归处理 $[l, m]$ 和 $[m+1, r]$ ，然后用 $f[l, r] = f[l, m] \odot f[m+1, r]$ 计算出本区间的结果。

查询. 对于查询 $[lq, rq]$ ，从 $[1, n]$ 开始分治。假设当前的分治区间是 $[l, r]$ ，中点为 $m = \lfloor (l+r)/2 \rfloor$ 。如果 $[lq, rq] = [l, r]$ ，那么直接返回预处理的 $f[l, r]$ ；如果 $[lq, rq]$ 完全在 $[l, m]$ 或者 $[m+1, r]$ ，则递归下去处理；否则，递归 $\text{query}(l, m, lq, m)$ 和 $\text{query}(m+1, r, m+1, rq)$ 来算出 $f[lq, m]$ 和 $f[m+1, rq]$ ，然后返回 $f[lq, m] \odot f[m+1, rq]$ 。

现在来看一下复杂度。预处理的复杂度是 $T(n) = 2T(n/2) + O(1) = O(n)$ ，而询问的复杂度初看似乎是 $Q(n) = 2Q(n/2) + O(1) = O(n)$ 。然而，根据我们先前的分析，在第一次两侧递归后，两个询问就都变成了前/后缀询问。而对于前/后缀询问，只有一侧需要递归计算。所以复杂度是 $Q(n) = Q(n/2) + O(1) = O(\log n)$ 。

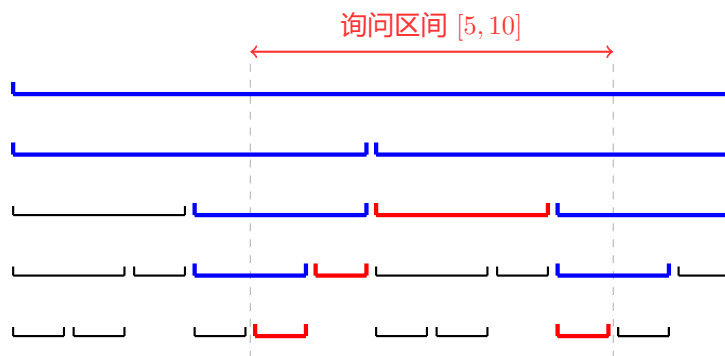


图 2.3: 线段树上定位区间

图 2.3 展示了在线段树上定位 $[5, 10]$ 的情况。红色的区间代表 $[lq, rq] = [l, r]$ 的终止区间，蓝色的区间代表需要到两侧递归询问的区间。结合上面对询问的分治过程的分析，我们可以得到以下定理：

定理 2.16. 区间在线段树上的分解. 设线段树树高为 $h \geq 3$ 。对任何询问区间，其在线段树上的分治过程中至多会访问 $2h - 3$ 个蓝色区间（需要进一步分治的区间）和 $2h - 4$ 个红色区间（可以直接返回的区间）。

评注. 常数 -3 和 -4 其实并不重要。但是系数 2 是重要的。

证明. 先考虑前/后缀询问的情况。除了最后一层，每层至多产生一个蓝色区间；除了第一层，每层至多产生一个红色区间。所以任何前/后缀会访问不超过 $h - 1$ 个蓝色区间和 $h - 1$ 个红色区间。

现在考虑一般的询问区间。在其分解为前后缀询问之前，每层恰好访问一个蓝色区间。所以最坏情况是在第一层就分解为前后缀询问，此时总共访问的蓝色区间数量不超过 $1 + 2(h - 2)$ ，红色区间数量不超过 $2(h - 2)$ 。□

使用上面的定理时需要知道树高。这是容易看出的：

性质 2.17. 线段树的树高. 按照 $m = \lfloor (l + r)/2 \rfloor$ 划分区间为 $[l, m]$ 和 $[m + 1, r]$ 得到的线段树高度为 $h(n) = h(\lceil n/2 \rceil) + 1 = 1 + \lceil \log_2 n \rceil$ 。

评注. 这里暗示了不均匀划分的线段树的存在。虽然不均匀划分一般没什么用，但是会有人拿来出题。

在实现线段树的代码之前，我们需要思考一个问题：如何方便地表示所有预处理的 $f[l, r]$ ？自然的想法是，对分治树的每个结点，在这个结点上记录其对应的区间的 f 。那么问题就转换为了如何存储这棵树。一般的做法当然是指针式的，即每个结点记录其左右儿子结点的编号（或者干脆就是指针）。不过在这里，我们可以类似二叉堆来编号，即让结点 u 的左右儿子分别编号为 $2u$ 和 $2u + 1$ 。图 2.2 中的分治树的结点编号如图 2.4 所示。

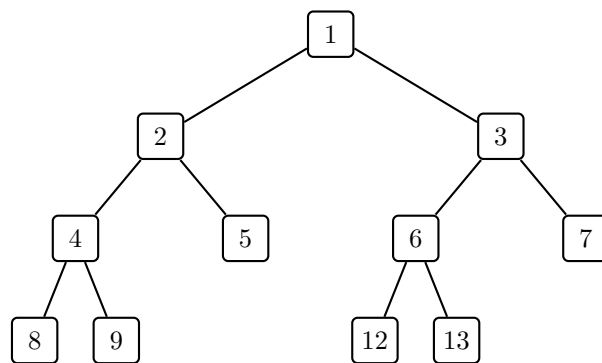


图 2.4: 图 2.2 中分治树的编号

有两个事情需要注意：第一，最后一层也就是叶子层的编号并不是连续的；第二，最大的编号可以达到 $4n - O(1)$ 级别，我们有一个练习来仔细计算这个值。

代码 2.4: 线段树

```

1  INFO tree[N<<2];
2  void build(int u, int l, int r) {
3      if (l == r) {

```



```

4         tree[u] = a[l];
5         return;
6     }
7     int mid = (l + r) >> 1;
8     build(u << 1, l, mid);
9     build(u << 1 | 1, mid + 1, r);
10    tree[u] = tree[u << 1] + tree[u << 1 | 1];
11 }
12 Info query(int u, int l, int r, int lq, int rq) {
13     if (l==lq && r==rq) {
14         return tree[u];
15     }
16     int mid = (l + r) >> 1;
17     if (rq <= mid) return query(u << 1, l, mid, lq, rq);
18     else if (lq > mid) return query(u << 1 | 1, mid + 1, r, lq, rq);
19     else return query(u<<1,l,mid,lq,mid)+query(u<<1|1,mid+1,r,mid+1,rq);
20 }

```

评注. 事实上, 线段树可以认为是在描述分治结构。在本讲义中, 我们有时会混用“线段树”和“分治树”, 即将线段树视为一种结构的描述, 而非专门的数据结构。

练习 2.18. 线段树的动态开点. 在本练习中, 我们说明可以对长度巨大但是“有意义的位置”很少的序列在合理的时空复杂度内构建线段树。形式化地说, 假设序列长度为 n , 其中只有 m 个位置上的元素被特殊指定, 其余位置上的元素由某种默认模式给出, 如默认置为某个固定元素 x_0 。对于一个不含特殊指定元素的区间, 需要能够在 $O(1)$ 时间内计算这个区间的 $f[l, r]$ 。

1. 堆式编号还能用吗? 应该如何存储树?
2. 在 `build` 中, 如果当前区间 $[l, r]$ 不含特殊指定元素, 那么可以直接计算此区间的 $f[l, r]$ 作为 `tree[u]` 并返回。事实上, 甚至不需要建立这个区间对应的结点。基于这一观察, 计算需要建立的结点的数量级。
3. 如何判断当前区间是否含有特殊指定元素? [提示](#)
4. 修改 `build` 和 `query`, 使得 `build` 的时空复杂度为 $O(m \log n)$, 并且 `query` 能在和原来相同的时间内回答询问。

评注. 可以通过压缩 *Trie* 进一步把空间降为 $O(m)$, 但是一般没有必要。

评注. 在下一节中, 我们会通过单点修改的方式完成相同的目标。

练习 2.19. 在本练习中, 我们给出一种从左到右动态构建分治树的算法, 从而让二区间合并和线段树都支持末尾插入。

想象我们有完整的长为 $n = 2^k$ 的序列, 然后在其上建立分治树。而实际上, 在任意时刻, 我们只拥有这个序列的一个前缀。在分治树上, 这个前缀对应“左下角”的一些部分。具

体来说，如果一个结点对应的区间中的元素都已经被插入，那么我们就称这个结点是完整的。我们的算法思想非常简单：在末尾插入一个元素后，把分治树上所有因为这个插入变得完整的结点构建出来。



图 2.5: 序列前缀在分治树上

1. 假设目前给出了序列的前 m 个元素。说明我们相当于在长为 $2^{\lceil \log_2 m \rceil}$ 的序列形成的分治树上操作。
2. 末尾插入 a_i 后，哪些结点会变得完整？
3. 说明这个算法在插入 q 次后花费的总时间不超过 $T(2^{\lceil \log_2 q \rceil})$ 的，其中 $T(m)$ 是对长为 m 的静态序列构建数据结构所需的时间。所以，在通常的情况下（也就是 $T(m) = m^{O(1)}$ ），每次插入的均摊复杂度不超过 $O(T(q)/q)$ 。分别对于二区间合并和线段树计算这个复杂度。

评注. *TODO* 插入与合并。

2.4 * 一般的静态区间数据结构